

# תרגול הנדסת תוכנה

## מבוא להנדסת תוכנה

הסיכום מסודר לפי נושאי הקורס — אותם נושאים שלפיהם מתויגות שאלות התרגול. הוא מיועד לחזרה מהירה לפני המבחן ולחיזוק ההבנה לצד התרגול. המונחים הלועזיים נשמרים באנגלית כפי שמופיעים במבחנים.

**הנדסת תוכנה (Software Engineering)** היא דיסציפלינה הנדסית העוסקת בכל היבטי ייצור התוכנה — מהגדרת הדרישות ועד לתחזוקה שלאחר האספקה. המטרה: לייצר תוכנה **איכותית**, בזמן ובתקציב.

- **Software Development מול Professional Software Development:** פיתוח מקצועי נעשה בדרך כלל בצוותים (לא ביחידים), ומחייב **תיעוד, ניהול תצורה, בדיקות ותהליכים** מסודרים — לא רק כתיבת קוד שעובד.
- **תוכנה = תוכניות + נתונים + תיעוד + קונפיגורציה** (לא רק קוד המקור).
- **שני סוגי מוצרי תוכנה:**
  - **Generic products** — מוצר מדף שהיצרן מוכר לשוק פתוח (Word, Photoshop). היצרן קובע את הדרישות.
  - **Customized / Bespoke products** — מוצר שמפותח ללקוח ספציפי לפי דרישותיו.
  - השוני בין השניים הולך ומיטשטש: מוצרים גנריים ניתנים להתאמה (configuration), ולקוחות בונים על תשתיות קיימות.
- **תכונות איכות (Quality attributes)** של תוכנה טובה: **Maintainability** (ניתנות לתחזוקה), **Dependability** & **Security** (אמינות ואבטחה), **Efficiency** (יעילות), **Acceptability** (קבילות למשתמש).
- **סוגי יישומים:** Stand-alone, Interactive transaction-based (web apps), Embedded, Batch, Entertainment, Data-collection, Systems-of-systems.
- **עקרונות-על:** להתבסס על **תהליך מוגדר**, לשים לב ל**אמינות ולביצועים**, להבין ולנהל את **הדרישות**, ולעשות שימוש חוזר (**reuse**) במקום לבנות מאפס.

## מודלים לתהליך פיתוח תוכנה

**Software Process** = אוסף הפעילויות המובילות לייצור מוצר תוכנה. ארבע פעילויות יסוד: **Specification** (הגדרה), **Design & Implementation** (עיצוב ומימוש), **Validation** (אימות), **Evolution** (התפתחות/תחזוקה).

- **Build and Fix** — כותבים קוד ומתקנים עד שעובד. אין שלבי תכנון/עיצוב מסודרים. מתאים רק לתוכניות זעירות; **קשה מאוד לתחזוקה והרחבה**.
- **Waterfall (מפל המים)** — שלבים **רציפים** ומוגדרים היטב: Requirements → Design → Implementation → Testing → Maintenance. כל שלב מסתיים לפני הבא.
- **יתרון:** שלבים מוגדרים ומתועדים → **קל לניהול ולתכנון**, מתאים כשהדרישות ידועות ויציבות.
- **חסרון: נוקשה** — קשה להגיב לשינויי דרישות; הלקוח רואה מוצר רק בסוף.
- **Incremental development (פיתוח מצטבר)** — בונים את המערכת בגרסאות (increments); כל increment מוסיף פונקציונליות. משוב מוקדם, זול יותר לשינוי. בסיס לשיטות Agile.
- **Reuse-oriented** — הרכבת המערכת מרכיבים (components) קיימים. מפחית עלות וסיכון, אך מגביל בהתאמה מדויקת לדרישות.

- **Prototyping** — בונים אב-טיפוס מהיר כדי להבהיר דרישות או לבדוק היתכנות (Proof of Concept). ה-prototype לרוב "נזרק" ואינו המוצר הסופי.
- **Host-Target model** — התוכנה **מפותחת** על מכונה אחת (host) אך **רצה** על אחרת (target). הפער מטופל ע"י: שפות מבוססות **מכונה וירטואלית** (Java/JVM — "כתוב פעם אחת, רוץ בכל מקום"), סימולטורים/אמולטורים, וכלי cross-compilation.

## XP, Agile, SCRUM

**Agile** = משפחת שיטות פיתוח **מצטברות ואיטרטיביות** המדגישות אספקת ערך מהירה ותגובה לשינוי. **The Agile Manifesto**: מעדיפים **אנשים ואינטראקציות** על תהליכים וכלים · **תוכנה עובדת** על תיעוד מקיף · **שיתוף** לקוח על משא-ומתן חוזי · **תגובה לשינוי** על מעקב אחר תוכנית.

### SCRUM

מסגרת (framework) לניהול פיתוח אגילי. עובדים ב-**Sprints** (איטרציות קצובות, לרוב 2-4 שבועות).

- **תפקידים (Roles)**:
- **Product Owner** — מגדיר את **חזון המוצר**, מנהל ומתעדף את ה-**Product Backlog**, מייצג את הלקוח.
- **Scrum Master** — מנהל/מנחה את **תהליך ה-Scrum**, מסיר חסמים, אינו "מנהל" קלאסי.
- **Development Team** — צוות רב-תחומי שמבצע את העבודה.
- **אירועים (Events)**:
- **Sprint Planning** — תכנון משימות הספרינט הבא.
- **Daily Scrum (Stand-up)** — פגישה **יומית** קצרה (~10-20 דק') בתחילת היום: מה עשיתי, מה אעשה, מה חוסם.
- **Sprint Review** — בסוף הספרינט: דיון בעבודה **שהושלמה** ועדכון ה-Backlog מול בעלי עניין.
- **Sprint Retrospective** — הצוות בוחן את **התהליך** ומשפר לספרינט הבא.
- **תוצרים (Artifacts)**: **Product Backlog** (כל הדרישות), **Sprint Backlog** (תת-קבוצה שנבחרה לספרינט), **Increment** (תוצר עובד).
- **User Stories** — תיאור דרישה מנקודת מבט המשתמש: "*As a <role>, I want <goal> so that <benefit>*".
- **שערוך עלויות** — באמצעות **Story Points** (בדיון, לרוב Planning Poker), ולא בשעות מדויקות.

### Extreme Programming (XP)

שיטת אגילי עם פרקטיקות הנדסיות: **Pair Programming, Test-Driven Development, Continuous Integration, Refactoring, Small Releases, Collective Ownership**. XP-I, **On-site Customer** מדגישה מקבילות של פיתוח ובדיקות (בסיס ל-TDD).

## הנדסת דרישות

**Requirement** = תיאור של **מה** המערכת צריכה לעשות (services) ושל **האילוץ** על פעולתה. אוספים, מנתחים, מתעדים ומאמתים.

- **Functional requirements** — מה המערכת עושה: שירותים, קלט/פלט, התנהגות.

- **Non-Functional Requirements (NFR)** — **אילוצי איכות** על המערכת כולה. דוגמאות: **Performance** (זמן תגובה  $\geq 1$  שני), **Security** (הצפנת TLS), **Reliability, Availability, Usability, Portability** (תאימות ל-Android/דפדפנים), **Scalability**. לרוב **קשים למדידה** אך קריטיים — כשל בהם עלול לפסול את המערכת כולה.
- **User requirements** (בשפת הלקוח) מול **System requirements** (מפורטים, טכניים).
- **מסמכים: MRD** (Market Requirements) — מה השוק צריך, נכתב ע"י המחלקה העסקית (PRD Product Requirements — מה המוצר יעשה, טכני יותר).
- **Elicitation techniques** (איסוף דרישות): ראיונות, שאלונים, תצפית, prototyping, **Use Cases, User Stories**, ניתוח מסמכים.
- **Cucumber / BDD (Behaviour-Driven Development)** — כתיבת דרישות כתרחישים בפורמט **Given-When-Then** (Gherkin), שמשמשים גם כבדיקות קבלה אוטומטיות.
- **דרישה טובה**: ברורה, חד-משמעית, שלמה, עקבית, ניתנת לאימות (verifiable) ולמעקב (traceable).

## מידול UML

**UML (Unified Modeling Language)** — שפת מידול גרפית סטנדרטית. מבחינים בין דיאגרמות **מבניות (structural)** לבין **התנהגותיות (behavioral)**.

- **Use Case Diagram** — מתאר את **האינטראקציה** בין **Actors** למערכת (מודל אינטראקציה). יחסים: **<<include>>** (התנהגות משותפת שתמיד מתבצעת), **<<extend>>** (התנהגות אופציונלית). כאשר יש הרבה use-cases אפשר לפצל למספר תרשימים.
- **Class Diagram** — דיאגרמה **מבנית** (structure model) המתארת מחלקות, תכונות, פעולות והקשרים ביניהן. נוצרת בשלב ה-**Design & Development**. יחסים:

| יחס            | סימון          | משמעות                                      |
|----------------|----------------|---------------------------------------------|
| Association    | קו             | קשר כללי בין מחלקות                         |
| Aggregation    | מעוין ריק ◇    | "has-a" — הכלה חלשה, החלקים שורדים בלי השלם |
| Composition    | מעוין מלא ◆    | הכלה חזקה — החלקים תלויי-חיים בשלם          |
| Generalization | חץ משולש ריק ▷ | ירשה "is-a"                                 |
| Dependency     | חץ מקווקו      | תלות זמנית                                  |

**ריבוי (Multiplicity)**: 1, 0..1, \*..1, \* קשר \*..1 ממומש לרוב ברשימה/אוסף.

- **Sequence Diagram** — דיאגרמה **התנהגותית** (interaction model): מתארת הודעות בין אובייקטים **לאורך ציר הזמן** (מלמעלה למטה). **Lifeline** = קו אנכי מקווקו; **Activation bar** = מלבן על ה-lifeline המסמן שהאובייקט **פעיל**; חץ מלא = קריאה, חץ מקווקו = תשובה/return.

- **State Diagram / Statechart** — דיאגרמה התנהגותית של **מצבים ומעברים**. מעבר מופעל ע"י אירוע ויכול לשאת **guard** (תנאי בסוגריים מרובעים). מצבים מורכבים: **XOR** (OR-state, מצב אחד פעיל בכל רגע) ו-**AND** (מצבים מקבילים בו-זמנית).
- **Activity Diagram** — תיאור זרימת עבודה/תהליך; משמש למשל במודל ה-Context.

## עיצוב ארכיטקטוני

**Software Architecture** — המבנה הכללי של המערכת: הרכיבים המרכזיים והקשרים ביניהם. ההחלטות הארכיטקטוניות משפיעות על תכונות האיכות (ביצועים, אבטחה, ניתנות-לתחזוקה).

- **תצוגות (Views) — מודל 4+1**: **Logical** (אובייקטים/מחלקות), **Process** (תהליכים ותקשורת), **Development** (מודולים/רכיבים), **Physical/Deployment** (חומרה), ו-**Use-case** שמחבר את כולן. אין תצוגה יחידה שמתארת הכל.

### Architectural patterns:

- **Layered (שכבות)** — כל שכבה משתמשת בשירותי השכבה שמתחתיה. **יתרון**: הפרדה, שינוי בשכבה משפיע מעט על האחרות, ניתנות להחלפה. **MVC** נחשב מקרה פרטי/קרוב.
- **MVC (Model-View-Controller)** — הפרדה בין **נתונים (Model)**, **תצוגה (View)** ו-**בקרה (Controller)**. מאפשר תצוגות מרובות ושינוי UI בלי לגעת בלוגיקה. (משמש גם כתבנית עיצוב).
- **Repository** — כל הרכיבים חולקים **מאגר נתונים מרכזי**. **יתרון**: שיתוף יעיל. **חסרון עיקרי**: המאגר המרכזי הוא נקודת כשל וצוואר-בקבוק — **קשה להפיץ (distribute) על פני מספר מכונות** ביעילות.
- **Client-Server** — שרתים מספקים שירותים ללקוחות דרך הרשת; לרוב שירות לכל שרת.
- **Pipe and Filter** — עיבוד נתונים בשרשרת שלבים (filters) שמעבירים נתונים ב-pipes. מתאים ל-batch/עיבוד נתונים, **לא** מתאים למערכות אינטראקטיביות.
- **מערכות מבוזרות (Distributed systems)** — מטפלות היטב ב-**Scalability** (Scale-Up/Out), אך מציבות אתגרים: **Security, Concurrency**, עקביות ו-fault tolerance.

## תבניות עיצוב (Design Patterns)

**Design Pattern** = פתרון גנרי מוכח לבעיית עיצוב חוזרת. GoF מחלקים לשלוש קטגוריות: **Creational** (יצירה), **Structural** (מבנה), **Behavioral** (התנהגות).

### Creational

- **Singleton** — מבטיח **מופע יחיד** למחלקה ונקודת גישה גלובלית אליו. מימוש: בנאי `private` + שדה `static` + `getInstance()`:

```
public class Config {
    private static Config instance;

    private Config() {}

    public static Config getInstance() {
        if (instance == null) instance = new Config();
        return instance;
    }
}
```

- **Simple Factory** — מחלקה שמרכזת יצירת אובייקטים לפי פרמטר (לא תבנית GoF רשמית, אך נלמדת).
- **Factory Method** — מגדיר **ממשק ליצירת אובייקט**, ומאפשר לתת-מחלקות להחליט **איזה** מחלקה קונקרטית ליצור. הפעולה מחזירה טיפוס-על/ממשק, המימוש מכריע את המופע.
- **Abstract Factory** — מפעל ליצירת **משפחות** של אובייקטים קשורים בלי לציין מחלקות קונקרטיות.

## Structural

- **Adapter** — עוטף מחלקה כדי להתאים את הממשק שלה לממשק שהלקוח מצפה לו ("מתאם חשמלי").
- **Decorator** — מוסיף אחריות לאובייקט **דינמית** ע"י עטיפה, בלי לשנות את המחלקה. חלופה גמישה לירושה.
- **Facade** — ממשק פשוט אחיד מעל תת-מערכת מורכבת; מסתיר את הפרטים מהלקוח.
- **Flyweight** — חוסך זיכרון ע"י **שיתוף** אובייקטים זהים (intrinsic state משותף, extrinsic מבחוץ). שימושי כשיש **מספר עצום** של אובייקטים דומים.
- **Composite** — מבנה עץ שבו אובייקט בודד וקבוצת אובייקטים מטופלים באחידות.
- **Proxy** — מיופה-כוח השולט בגישה לאובייקט אחר (lazy loading, הרשאות).

## Behavioral

- **Observer** — מגדיר תלות **one-to-many**: כאשר ה-**Subject** משנה מצב, כל ה-**Observers** המנויים מקבלים עדכון ( `notify` ). ה-**Subject** מחזיק רשימת **observers** (aggregation) ומפעיל עליהם `update`.
- **Strategy** — מכמס משפחת אלגוריתמים מאחורי ממשק ומאפשר החלפה בזמן ריצה.
- **Command** — עוטף בקשה כאובייקט. משתתפים: **Invoker** (מפעיל), **Receiver** (מבצע בפועל), **Command** (ממשק `execute`), **ConcreteCommand**.
- **Invoker** (מבצע בפועל), **Receiver** (מפעיל). **מנתק (decouples)** את המפעיל מהמבצע; מאפשר `undo/queue`.
- **Iterator** — גישה סדרתית לאיברי אוסף בלי לחשוף את מבנהו הפנימי ( `hasNext / next` ).
- **Memento** — שומר ומשחזר מצב פנימי של אובייקט (`undo`) בלי להפר כימוס.
- **Mediator** — מרכז את התקשורת בין אובייקטים, מפחית תלות ישירה ("many-to-many" → "one-to-many"). ההבדל מ-**Observer**: **Observer** הוא תלות מצב **one-to-many**; **Mediator** מתווך אינטראקציות רב-כיווניות.
- **MVC** — נחשב גם תבנית עיצוב (הפרדת **Model/View/Controller** עם **Observer** בין **Model** ל-**View**).

## עקרונות עיצוב ו-SOLID

עיצוב טוב שואף ל-**Coupling נמוך** (תלות מעטה בין מודולים) ו-**Cohesion גבוה** (כל מודול בעל אחריות ממוקדת).

- **SOLID**:
- **S — Single Responsibility**: למחלקה **סיבה אחת** להשתנות (אחריות אחת).
- **O — Open/Closed**: **פתוח להרחבה, סגור לשינוי** — מוסיפים התנהגות בלי לשנות קוד קיים (בד"כ דרך ירושה/ממשקים).
- **L — Liskov Substitution (LSP)**: אובייקט של תת-מחלקה חייב להיות בר-החלפה עם אובייקט מחלקת-העל **בלי לשבור** את נכונות התוכנית. תת-מחלקה לא תחזק דרישות-קדם ולא תחליש דרישות-אחר.
- **I — Interface Segregation**: עדיף ממשקים **קטנים וממוקדים** על ממשק "שמן" אחד; לקוח לא יאולץ לתלות בשיטות שאינו צריך.
- **D — Dependency Inversion**: תלות ב-**אבסטרקציות** (ממשקים) ולא במימושים קונקרטיים.
- **DRY (Don't Repeat Yourself)** — להימנע משכפול ידע/קוד.
- **Refactoring** — שיפור מבנה הקוד בלי לשנות התנהגות חיצונית (נשען על בדיקות).

## בדיקות תוכנה, TDD ו-JUnit

**Verification & Validation (V&V): Verification** — "האם בנינו את המוצר **נכון?**" (מול המפרט). **Validation** — "האם בנינו את המוצר הנכון?" (מול צורכי הלקוח).

• **רמות/סוגי בדיקות:**

- **Unit Test** — בדיקת יחידה (מחלקה/מתודה) בבידוד.
- **Integration Test** — בדיקת האינטראקציה בין יחידות.
- **System Test** — בדיקת המערכת השלמה.
- **Acceptance Test** — אימות מול הלקוח/הדרישות.
- **Inspection / Static testing** — סקירת קוד **ללא הרצה** (static). מתבצעת **מוקדם** ולפני בדיקות דינמיות.
- **תכנון מקרי בדיקה: Equivalence Partitioning** (חלוקה למחלקות שקילות), **Boundary Value Analysis** (בדיקת ערכי-גבול — שם באגים מתחבאים), ובדיקת מקרי קצה וקלטים לא סבירים.
- **TDD (Test-Driven Development)** — מחזור **Red → Green → Refactor**: כותבים **בדיקה כושלת** קודם, אז קוד מינימלי שמעביר אותה, ואז מבצעים refactor. מקדם **מקביליות של פיתוח ובדיקות** ותכנון ממשקים טוב. פרקטיקה מרכזית ב-XP.
- **JUnit** — framework לבדיקות יחידה ב-Java (בגרסאות ישנות @Before / @After; @BeforeAll / @AfterAll רצות פעם אחת לכל המחלקה):

```
@BeforeEach void setUp() { /* רץ לפני כל בדיקה */ }
@AfterEach void tearDown(){ /* רץ אחרי כל בדיקה */ }
@Test void addsTwoNumbers() {
    assertEquals(5, calc.add(2, 3)); // assertEquals / assertTrue / assertNull /
    assertThrows
}
```

- **Mock Objects** — אובייקטים מזויפים המחליפים של היחידה-הנבדקת (DB, שירות רשת), כדי לבודד אותה ולשלוט בהתנהגות התלות. מסגרת נפוצה: **Mockito**. מבחינים בין Mock/Stub/Fake.
- **Coverage** — אחוז הקוד שהבדיקות מריצות (statement/branch coverage); כיסוי גבוה אינו מבטיח נכונות.

## תכנות מונחה עצמים ו-Java

- **OOP** — ארבעה עקרונות: **Encapsulation** (כימוס), **Inheritance** (ירושה), **Polymorphism** (רב-צורתיות), **Abstraction** (הפשטה).
- **Class מול Object** — מחלקה היא תבנית; אובייקט הוא מופע.
- **Interface מול Abstract Class**: מגדיר **חזוה** (שיטות ללא מימוש; מרובה ירושת-ממשק); **Abstract class** יכולה להכיל מימוש חלקי ושדות, אך ירושה יחידה.
- **Polymorphism** — קריאה לאותה שיטה מתנהגת שונה לפי הטיפוס הדינמי (override); overloading = אותו שם, חתימות שונות.
- **Generics** — טיפוסים פרמטריים (`<List<T>`) לבטיחות-טיפוסים בזמן קומפילציה.
- **Collections** — `List` (`ArrayList/LinkedList`), `Set`, `Map`, `Queue` ו-`Iterator`.
- **Exceptions** — `checked` מול `unchecked`; `try/catch/finally`.
- **Reflection** — יכולת התוכנית לבחון ולשנות את עצמה בזמן ריצה: לגלות מחלקות, שיטות ושדות דרך ה-`Class`, `getMethods()`, `getFields()`, `getConstructors()`, `invoke` **חיסרון: לא יעילה** (עוקפת אופטימיזציות, יוצרת/בוחנת אובייקטים רבים בזמן ריצה) ופוגעת בבטיחות-טיפוסים. שימושים: `frameworks`, סריאליזציה, `IoC/DI`.

## ניהול קונפיגורציה ואספקת תוכנה

- **Software Configuration Management (SCM)** — ניהול השינויים בתוצרי הפרויקט לאורך זמן.
- **Version Control (בקרת גרסאות)** — מעקב אחר שינויים בקוד. פעולות: **commit**, **branch** (ענף פיתוח מקבילי), **merge** (מיזוג ענפים), **git checkout** הוא הכלי הנפוץ.
- **ענף (branch)** מכיל את ההתקדמות שבוצעה בו; **merge** מאחד את ההתקדמות של ענף אחד לתוך אחר. ענף שממשיך להתפתח עצמאית לאחר מיזוג מכיל התקדמות שאין בענפים אחרים.
- **Centralized (למשל SVN)** — מאגר מרכזי אחד; **קל לניהול** אך תלוי בשרת.

- **Distributed (git)** — לכל מפתח עותק מלא; מאפשר **commit/rollback** מקומי ועבודה offline.
- **Continuous Integration (CI)** — מיזוג ובנייה תכופים של הקוד המשולב, עם הרצת בדיקות אוטומטית → גילוי מוקדם של תקלות אינטגרציה.
- **Continuous Delivery מול Continuous Deployment**:
- **Continuous Delivery** — כל build שעובר בדיקות מוכן לאספקה, אך הפריסה לפרודקשן ידנית/מאושרת.
- **Continuous Deployment** — כל build שעובר בדיקות נפרס אוטומטית לפרודקשן.
- **Streamlining Software Delivery** — אוטומציה של pipeline הבנייה-בדיקה-אספקה (DevOps) לקיצור זמן וסיכון.

## מטריקות ותצוגות

- **Software Metrics** — מדידה כמותית של תכונות התוכנה או התהליך, כדי לנהל, לחזות ולשפר איכות.
- **מטריקות מוצר: LOC** (שורות קוד), **Cyclomatic Complexity** (מורכבות מסלולים), עומק ירושה, coupling/cohesion, מספר פונקציות.
- **מטריקות תהליך**: קצב תיקון תקלות, זמן פיתוח, effort.
- **עיקרון**: "You can't manage what you can't measure" — אך יש להיזהר ממדדים שמעודדים התנהגות שגויה (LOC אינו מדד לפרודוקטיביות).
- **Views (תצוגות)** — מציגות היבטים שונים של המערכת לבעלי-עניין שונים (ראו מודל 4+1 לעיל); אין תצוגה אחת שמכסה את הכל.

## טבלאות קיצור מהירות

### תבניות עיצוב לפי קטגוריה:

| קטגוריה    | תבניות                                                          |
|------------|-----------------------------------------------------------------|
| Creational | Singleton, Factory Method, Abstract Factory, Simple Factory     |
| Structural | Adapter, Decorator, Facade, Flyweight, Composite, Proxy         |
| Behavioral | Observer, Strategy, Command, Iterator, Memento, Mediator, (MVC) |

### מתי איזו דיאגרמת UML:

| רוצים לתאר           | דיאגרמה    | סוג                     |
|----------------------|------------|-------------------------|
| מי משתמש במערכת ואיך | Use Case   | אינטראקציה              |
| מבנה מחלקות וקשרים   | Class      | מבנית (structure)       |
| הודעות לאורך זמן     | Sequence   | התנהגותית (interaction) |
| מצבים ומעברים        | Statechart | התנהגותית               |
| זרימת תהליך/עבודה    | Activity   | התנהגותית               |

מפתח מהיר לבדיקות:

| פירוש                              | מונח         |
|------------------------------------|--------------|
| בנינו נכון? (מול המפרט)            | Verification |
| בנינו את הנכון? (מול הלקוח)        | Validation   |
| Red → Green → Refactor             | TDD          |
| רץ לפני כל בדיקה                   | BeforeEach@  |
| תחליף מבודד לתלות של היחידה הנבדקת | Mock         |
| בדיקה סטטית ללא הרצה               | Inspection   |

הסיכום נכתב לחזרה לפני המבחן ולחיזוק התרגול. בכל מקרה של סתירה — חומר הקורס (מצגות אקסמן/ניסים והסיכומים) הוא המקור הקובע.